

conseguIR Vignette: SLE, Lymphoid Mutations, and DLBCL Epigenomics

The consequiR authors

April 21, 2026

```
library(conseguIR)
library(data.table)
```

1 Introduction

This vignette shows a lightweight worked analysis with `conseguIR` using three modalities:

- germline risk from an SLE GWAS
- somatic mutation enrichment from a PCAWG lymphoid-malignancy MAF with CLL excluded
- regulatory variability from three DLBCL H3K27ac bigWig tracks

The purpose of the vignette is to document how the package is used and how the three score branches fit together. The vignette keeps the epigenomic branch small on purpose so that the workflow remains readable and reproducible.

At a high level, the workflow has six stages:

1. validate the raw inputs
2. compute germline, somatic, and epigenomic scores
3. impose those scores on the gene-regulatory graph
4. run diffusion to obtain post-diffusion gene scores
5. extract a smaller selected subgraph
6. inspect the results through score plots, network plots, and a locus plot

Because some compute stages are intentionally heavy, the code below is shown as runnable analysis code but is not evaluated during document render. That keeps the vignette light while still recording the exact package calls used in the worked example.

2 Example inputs

The example analysis uses the following files.

```
gwas_path <- "data/raw/Testing/GCST90436637.h.tsv"
somatic_maf_path <- "data/raw/Testing/2026-01-09_no_CLL_lymph_only_pcastg_maf_tcga_order_hg38.maf"
reg_ref_path <- "data/processed/GRCh38-cCREs.loc"
refdb_path <- "data/raw/Testing/RefCDS_human_GRCh38.p12.rda"
```

```

reference_bfile <- "data/raw/g1000_eur/g1000_eur"
aid_covariate_path <- "data/raw/Testing/2026-01-26_all_reg_elems_sample_level_mut_frac_comparison_bet_on
sample_size <- 399917L

bw_files <- c(
  "data/raw/Testing/SRR1020516_DLBCL_P286_H3K27ac_ChIPseq.bw",
  "data/raw/Testing/SRR1554599_DLBCL_DHL6_H3K27ac_ChIPseq.bw",
  "data/raw/Testing/SRR7758629_DLBCL_U2932_H3K27ac_ChIPseq.bw"
)

data.table(
  object = c(
    "GWAS",
    "Somatic MAF",
    "Regulatory reference",
    "dndscv reference",
    "PLINK reference",
    "fishHook covariate data",
    "Selected bigWig tracks"
  ),
  value = c(
    gwas_path,
    somatic_maf_path,
    reg_ref_path,
    refdb_path,
    reference_bfile,
    aid_covariate_path,
    paste(basename(bw_files), collapse = ", ")
  )
)
#>           object
#>           <char>
#> 1:           GWAS
#> 2:       Somatic MAF
#> 3:  Regulatory reference
#> 4:    dndscv reference
#> 5:      PLINK reference
#> 6: fishHook covariate data
#> 7: Selected bigWig tracks
#>
#>
#> 1:
#> 2:                                     data/raw/Testing/2026-01-09_no_CLL_lymph_on
#> 3:
#> 4:
#> 5:                                     data/raw/Tes
#> 6: data/raw/Testing/2026-01-26_all_reg_elems_sample_level_mut_frac_comparison_bet_only_memory_b_norm
#> 7:      SRR1020516_DLBCL_P286_H3K27ac_ChIPseq.bw, SRR1554599_DLBCL_DHL6_H3K27ac_ChIPseq.bw, SRR775862

```

For the vignette we intentionally keep the epigenomic branch small. The three DLBCL tracks above were chosen to give a clean lymphoma-centered example while still making the cross-track variability step easy to understand.

3 Step 1: Validate the Inputs

Before running any heavy computation, we usually make sure the basic file and column structure is sound.

```
validation <- validate_inputs(  
  gwas_sumstats = gwas_path,  
  somatic_maf = somatic_maf_path,  
  reg_ref_path = reg_ref_path,  
  epigenomic_tracks = bw_files,  
  verbose = TRUE  
)
```

The validation stage is intentionally lightweight. It is meant to catch:

- missing files
- missing key columns
- unreadable regulatory references
- unreadable bigWig headers

It is not meant to fully dry-run MAGMA, dndscv, fishHook, or diffusion.

4 Step 2: Compute the Three Score Modalities

4.1 Germline scoring with MAGMA

The germline branch uses MAGMA twice, once for genes and once for regulatory elements. In this worked example the input GWAS is an SLE summary-statistics table, so the germline layer represents inherited autoimmune risk rather than a lymphoma-only signal.

```
germline_scores <- prepare_germline_scores(  
  gwas_sumstats = gwas_path,  
  reference_bfile = reference_bfile,  
  gene_sample_size = sample_size,  
  reg_sample_size = sample_size,  
  gene_step1_args = list(annotation_window = c(35, 10)),  
  gene_step2_args = list(  
    gene_model = "snp-wise=mean",  
    pval_use = c("SNP", "P"),  
    pval_duplicate = "drop"  
  ),  
  reg_step1_args = list(annotation_window = c(0, 0)),  
  reg_step2_args = list(  
    gene_model = "snp-wise=mean",  
    pval_use = c("SNP", "P"),  
    pval_duplicate = "drop"  
  ),  
  verbose = TRUE  
)
```

The key MAGMA idea is:

- step 1 decides which SNPs belong to each feature
- step 2 decides how SNP-level signal becomes one feature-level score

4.2 Somatic scoring with dndscv and fishHook

The somatic branch is split into a gene model and a regulatory model.

In the current package defaults, the gene branch asks `dndscv` for one-sided positive and negative selection output when that API surface is available, and the regulatory branch uses `fishHook`'s directional `p` and `p.neg` columns when present. That keeps score extraction aligned with the underlying statistical tests rather than reconstructing direction from a generic two-sided p-value.

```
covariate_dt <- readRDS(aid_covariate_path)

somatic_scores <- prepare_somatic_scores(
  maf = somatic_maf_path,
  refdb = refdb_path,
  reg_ref_path = reg_ref_path,
  dndscv_args = list(sm = "192r_3w"),
  fishhook_covariate_data = covariate_dt,
  fishhook_args = list(),
  verbose = TRUE
)
```

For this vignette the `fishHook` covariate is a precomputed **AID-per-base** background measure supplied in a ready-to-use table. Biologically, it represents expected local exposure to activation-induced cytidine deaminase (AID) mutagenesis. That matters in B-cell malignancies because some regulatory elements accumulate mutations partly because they sit in an AID-exposed background, not necessarily because they are under stronger positive selection.

Using a precomputed covariate keeps the vignette light: `fishHook` receives a table that is already aligned to the regulatory-element universe instead of recomputing sequence or sample-level summaries during document execution.

The practical table shape is still simple:

```
head(covariate_dt[, c("reg_elem_id", "n_samples_mut_norm")])

covariate_dt <- data.frame(
  reg_elem_id = c("EH38E0080197", "EH38E2084302"),
  n_samples_mut_norm = c(3.8, 0.9)
)
```

That is, one row per regulatory element, a regulatory-element identifier, and the covariate values `fishHook` should model. Inside `consequiR`, that table is converted into a `fishHook`-ready AID-per-base covariate after matching each regulatory element back onto the ENCODE cCRE universe.

4.3 Epigenomic scoring from bigWig tracks

The epigenomic score in `consequiR` is best interpreted as a **cross-track variability score**, not a generic activity score. For the vignette we keep this branch intentionally small and DLBCL-centered by using three H3K27ac tracks:

- SRR1020516_DLBCL_P286_H3K27ac_ChIPseq.bw
- SRR1554599_DLBCL_DHL6_H3K27ac_ChIPseq.bw
- SRR7758629_DLBCL_U2932_H3K27ac_ChIPseq.bw

This makes the example easier to follow than a much larger track panel while still preserving the idea that regulatory elements are being ranked by how much their activity changes across related malignant contexts.

```
epigenomic_scores <- prepare_epigenomic_scores(  
  reg_ref_path = reg_ref_path,  
  bw_files = bw_files,  
  min_tracks = 3L,  
  drop_mhc = TRUE,  
  transform = "log1p",  
  summary_fun = mean,  
  verbose = TRUE  
)
```

5 Step 3: Build the Scored Gene-Regulatory Graph

```
scored_graph <- build_scored_gene_reg_graph(  
  germline_scores = germline_scores,  
  somatic_scores = somatic_scores,  
  epigenomic_scores = epigenomic_scores,  
  verbose = TRUE  
)
```

At this point, all three modalities have been imposed onto the graph. Missing modalities are zero-filled by default, which keeps the graph broad enough to retain biologically meaningful features even when one branch is absent.

6 Step 4: Diffusion

As in the overview vignette, the diffusion and selected-subgraph stages are Python-backed internally but are run through `consequiR`'s managed `basilisk` environment rather than a user-supplied interpreter.

```
diffusion <- run_gene_reg_diffusion(  
  scored_graph = scored_graph,  
  top_k = 3L,  
  confidence_power = 2.0,  
  beta_germline = 0.5,  
  beta_somatic = 0.5,  
  beta_epigenomic = 0.7,  
  integration_weight_germline = 1.0,  
  integration_weight_somatic = 1.0,  
  integration_weight_epigenomic = 1.0,  
  reg_signal_clip = 5.0,  
  verbose = TRUE  
)
```

This stage turns the pre-diffusion graph scores into a gene-centered post-diffusion ranking that incorporates regulatory context.

The main ranking columns now use a weighted signed Stouffer-style integration across the three post-diffusion modalities. That means positive germline, somatic, and epigenomic support reinforce one another, while negative modality contributions are carried forward and can reduce the final integrated score.

7 Step 5: Selected Subgraph Calling

```
selected_subgraph <- call_selected_subgraph(  
  diffusion = diffusion,  
  target_genes = 50L,  
  candidate_pool_size = 400L,  
  node_prize_weight = 1,  
  edge_conf_weight = 1,  
  edge_cost_weight = 1,  
  verbose = TRUE  
)
```

This is the step where the workflow shifts from a full graph to a smaller, more interpretable network.

8 Step 6: Plotting the Results

The plotting functions below are the ones used to inspect each stage of the workflow. In this vignette they are shown as code only.

The previously embedded figure artifacts have been removed here because the checked-in PDFs on disk predate the current ENCODE-backed SLE/DLBCL worked example and should not be presented as if they were generated from this exact setup.

8.1 Germline gene plot

```
plot_germline_gene_scores(  
  germline_scores = germline_scores,  
  stage = "pre",  
  label_features = c("MYC", "BCL2", "CXCR5"),  
  plot_file_path = "germline_genes_pre_rank.pdf"  
)
```

This plot gives a quick read on which genes are leading the germline signal before diffusion.

8.2 Somatic gene plot

```
plot_somatic_gene_scores(  
  somatic_scores = somatic_scores,  
  stage = "pre",  
  label_features = c("MYC", "BCL2", "TP53"),  
  plot_file_path = "somatic_genes_pre_volcano.pdf"  
)
```

This is where we can see which genes dominate the somatic branch before regulatory context is allowed to propagate by diffusion.

8.3 Epigenomic regulatory plot

```
plot_epigenomic_reg_scores(  
  epigenomic_scores = epigenomic_scores,  
  label_features = c("NAMPT"),  
  plot_file_path = "epigenomic_regs.pdf"  
)
```

This figure emphasizes the regulatory elements that are most variable across the selected DLBCL epigenomic tracks.

8.4 Selected subgraph plot

```
plot_selected_subgraph(  
  selected_subgraph = selected_subgraph,  
  plot_file_path = "selected_subgraph.pdf"  
)
```

This is usually the point where the workflow becomes easiest to discuss with other people, because we have moved from long ranked tables to a smaller and more interpretable network view.

8.5 Locus-level plot

```
plot_locus_context(  
  chromosome = "7",  
  start = 106200000,  
  end = 106330000,  
  scored_graph = scored_graph,  
  diffusion = diffusion,  
  gwas_sumstats = gwas_path,  
  label_top_lit_snps = 3L,  
  label_features = c("NAMPT"),  
  plot_file_path = "SLE_NAMPT_locus_context.pdf",  
  verbose = TRUE  
)
```

The locus plot is where the package becomes especially useful for biological interpretation. In this worked example it is the natural place to inspect the **NAMPT** neighborhood while keeping the three input modalities in one view. It lets us connect:

- regulatory-element scores
- post-diffusion gene scores
- regulatory interactions
- and cited SNPs that overlap the locus

In this vignette, **NAMPT** is used as a motivating gene for interpretation rather than as a forced endpoint. The germline branch asks whether inherited autoimmune risk accumulates in the region, the somatic branch asks whether lymphoid mutations are enriched beyond background expectation, and the epigenomic branch asks whether the linked regulatory landscape is variable across malignant DLBCL states.

9 Closing Remarks

The important pattern across the workflow is that each stage adds a different layer of structure:

- input validation establishes trust in the raw data
- scoring provides modality-specific signal
- graph construction puts all modalities in one shared space
- diffusion makes the regulatory context gene-centered
- subgraph calling distills the large network into something more interpretable
- plotting turns those results into views that can actually support discussion

In practice, that is why **consequiR** works best as a staged workflow rather than as a one-line black box.

10 Session information

```
sessionInfo()
#> R version 4.5.2 (2025-10-31)
#> Platform: aarch64-apple-darwin20
#> Running under: macOS Sonoma 14.6.1
#>
#> Matrix products: default
#> BLAS: /System/Library/Frameworks/Accelerate.framework/Versions/A/Frameworks/vecLib.framework/Versions/A/vecLib.framework/Versions/A/vecLib
#> LAPACK: /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRlapack.dylib; LAPACK v
#>
#> locale:
#> [1] C
#>
#> time zone: America/New_York
#> tzcode source: internal
#>
#> attached base packages:
#> [1] stats graphics grDevices utils datasets methods base
#>
#> other attached packages:
#> [1] data.table_1.18.2.1 conseqR_0.0.1
#>
#> loaded via a namespace (and not attached):
#> [1] BSgenome.Hsapiens.UCSC.hg38_1.4.5 SummarizedExperiment_1.40.0
#> [3] gtable_0.3.6 rjson_0.2.23
#> [5] xfun_0.56 ggplot2_4.0.2
#> [7] htmlwidgets_1.6.4 ggrepel_0.9.6
#> [9] Biobase_2.70.0 lattice_0.22-7
#> [11] vctrs_0.7.1 tools_4.5.2
#> [13] bitops_1.0-9 generics_0.1.4
#> [15] stats4_4.5.2 curl_7.0.0
#> [17] parallel_4.5.2 tibble_3.3.1
#> [19] pkgconfig_2.0.3 Matrix_1.7-4
#> [21] BSgenome_1.78.0 RColorBrewer_1.1-3
#> [23] cigarillo_1.0.0 S7_0.2.1
#> [25] S4Vectors_0.48.0 lifecycle_1.0.5
```



```

#> [27] compiler_4.5.2
#> [29] dndscv_0.0.1.0
#> [31] Biostrings_2.78.0
#> [33] codetools_0.2-20
#> [35] htmltools_0.5.9
#> [37] yaml_2.3.12
#> [39] plotly_4.12.0
#> [41] crayon_1.5.3
#> [43] tidyr_1.3.2
#> [45] BiocParallel_1.44.0
#> [47] abind_1.4-8
#> [49] digest_0.6.39
#> [51] purrr_1.2.1
#> [53] ade4_1.7-24
#> [55] grid_4.5.2
#> [57] cli_3.6.5
#> [59] S4Arrays_1.10.1
#> [61] XML_3.99-0.22
#> [63] UCSC.utils_1.6.1
#> [65] XVector_0.50.0
#> [67] matrixStats_1.5.0
#> [69] otel_0.2.0
#> [71] png_0.1-8
#> [73] evaluate_1.0.5
#> [75] GenomicRanges_1.62.1
#> [77] BiocIO_1.20.0
#> [79] rtracklayer_1.70.1
#> [81] Rcpp_1.1.1
#> [83] BiocGenerics_0.56.0
#> [85] R6_2.6.1
#> [87] GenomicAlignments_1.46.0
#> [89] fishHook_0.1
farver_2.1.2
Rsamtools_2.26.0
Seqinfo_1.0.0
GenomeInfoDb_1.46.2
RCurl_1.98-1.17
lazyeval_0.2.2
pillar_1.11.1
seqinr_4.2-36
MASS_7.3-65
DelayedArray_0.34.1
tidyselect_1.2.1
dplyr_1.2.0
restfulr_0.0.16
fastmap_1.2.0
SparseArray_1.10.8
magrittr_2.0.4
dichromat_2.0-0.1
scales_1.4.0
rmarkdown_2.30
httr_1.4.8
igraph_2.2.2
reticulate_1.45.0
zoo_1.8-15
knitr_1.51
IRanges_2.44.0
viridisLite_0.4.3
rlang_1.1.7
glue_1.8.0
jsonlite_2.0.0
MatrixGenerics_1.22.0
gUtils_0.2.0

```